mdv build plan: chapters, stages and to-do list

A working guide for building a small markdown editor with a live split view: the stages, a master checklist, and short code examples for the engine and the renderer. Examples are in TypeScript, but every idea carries over to Rust, Go or Python.

1. Project overview

mdv is a tiny editor. Raw markdown lives on the left, the rendered result on the right, updated live. There is no toolbar, no sidebar and no clutter. Everything is a keystroke away, and files are plain `.md` on disk.

The rough plan from your notes:

1. Split view with a shared scroll position
2. Open and save files from the command line
3. Export the preview to PDF

The guiding rule: keep it small. If a feature needs a menu, rethink the feature.

Not in v1: plugins, cloud sync, WYSIWYG editing, a theme marketplace.

The app has three parts, and keeping them separate is the most important design decision:

- **Engine**: markdown text in, a tree (AST) out.
- **Renderer**: the tree in, HTML out.
- **Shell**: the window, the two panes, files, keybindings and export.

2. Roadmap and master to-do list

Stage	Goal	Done when	Rough effort
0. Foundations	Repo, tooling, fixtures	`test` and `build` commands run	1 day
1. Block parser	Text to block nodes	Headings, lists, quotes and code parse	3 days
2. Inline parser	Text inside blocks	Emphasis, code spans and links parse	2 days
3. Renderer	Tree to HTML	Output matches the mapping table	2 days
4. Split-view shell	Two live panes	Typing updates the preview	3 days
5. Files and CLI	Open, save, quit	`mdv notes.md` round-trips a file	2 days
6. PDF export	Preview to PDF	One keystroke produces a clean PDF	1 day
7. Testing	Confidence	Spec examples run in CI	2 days

Stage	Goal	Done when	Rough effort
8. Polish and release	Ship it	Installer and icon done	2 days

Effort is a rough guess for one person working part time on each stage. Adjust it after Stage 1, which is the hardest to estimate.

Stage 0: Foundations

- [ ] Pick the stack (language, UI toolkit, test runner)
- [ ] Create the repo with build, test and format commands
- [ ] Write a ten-line spec of what v1 does and does not do
- [ ] Collect 20 real markdown files as fixtures

Stage 1: Block parser

- [ ] Define the AST types
- [ ] Parse headings, paragraphs and horizontal rules
- [ ] Parse fenced code blocks
- [ ] Parse block quotes (recursive)
- [ ] Parse ordered and unordered lists
- [ ] Parse tables (GFM extension)

Stage 2: Inline parser

- [ ] Code spans first, so their contents are left alone
- [ ] Strong and emphasis
- [ ] Links and images
- [ ] Backslash escapes and autolinks
- [ ] Strikethrough

Stage 3: Renderer

- [ ] Escape text and attribute values
- [ ] Render every node type
- [ ] Block dangerous URLs such as `javascript:`
- [ ] Tight versus loose lists
- [ ] Add `data-line` attributes for scroll sync

Stage 4: Split-view shell

- [ ] Two panes with a fixed divider
- [ ] Re-render on input with a short debounce
- [ ] Shared scroll position
- [ ] Status bar: mode badge, file name, word count, line and column
- [ ] Keybindings: save, toggle preview, swap panes, quit

Stage 5: Files and command line

- [ ] `mdv file.md` opens the file, or starts an empty buffer if it does not exist
- [ ] Atomic save (write a temp file, then rename)
- [ ] Unsaved marker and a guard when quitting
- [ ] Detect external changes to the file
- [ ] Handle UTF-8 and CRLF line endings

Stage 6: PDF export

- [ ] Print stylesheet for the preview
- [ ] Export command (keystroke and CLI flag)
- [ ] Sensible page breaks and wrapped code blocks

Stage 7: Testing and hardening

- [ ] Unit tests for every rule in chapter 7
- [ ] Run CommonMark spec examples and track the pass rate
- [ ] Fuzz test: random input must never crash the parser
- [ ] Check the app stays responsive on a 1 MB file

Stage 8: Polish and release

- [ ] Hashi icon exported at all sizes (`.icns` and `.ico`)
- [ ] Dark theme first, light theme second
- [ ] README with a keystroke cheat sheet
- [ ] Package and publish (dmg, AppImage, installer)

3. Architecture

Data flows one way, through small pure functions:

```
source text --parseBlocks--> block tree --(inline pass)--> full AST --render--> HTML string --> preview pane
```

Why this split works:

- The parser and renderer know nothing about windows or files, so they are trivial to unit test.
- You can render to HTML now and to something else later (terminal, PDF) by swapping the renderer only.
- The shell only calls `render(parse(text))` and shows the result.

Suggested folder layout:

```
src/
  engine/   ast.ts  blocks.ts  inline.ts
  render/   html.ts
  shell/    app.ts  keys.ts  files.ts  export.ts
```

```
test/
  fixtures/  rules.test.ts  spec.test.ts
```

4. Stage 1: the engine, block parser

Goal: turn lines of text into block nodes.

Steps

1. Write the AST types first. Every later step depends on them.
2. Split the source into lines and walk them with an index.
3. At each line, check block starts in a fixed order: blank line, fenced code, heading, horizontal rule, quote, list, and finally paragraph as the fallback.
4. For containers (quotes, list items), strip the prefix and call `parseBlocks` again on the inner text.

The AST:

```ts
type Inline =
  | { type: 'text'; value: string }
  | { type: 'em'; children: Inline[] }
  | { type: 'strong'; children: Inline[] }
  | { type: 'code'; value: string }
  | { type: 'link'; href: string; children: Inline[] };

type Block =
  | { type: 'heading'; level: 1 | 2 | 3 | 4 | 5 | 6; children: Inline[]; line: number }
  | { type: 'paragraph'; children: Inline[]; line: number }
  | { type: 'list'; ordered: boolean; items: Block[][]; line: number }
  | { type: 'quote'; children: Block[]; line: number }
  | { type: 'code'; lang: string; value: string; line: number }
  | { type: 'hr'; line: number };
```

The `line` field costs nothing now and makes scroll sync accurate later.

A minimal block parser:

```ts
const FENCE = '`'.repeat(3);

function parseBlocks(src: string, base = 0): Block[] {
  const lines = src.split('\n');
  const out: Block[] = [];
  let i = 0;

  while (i < lines.length) {
    const line = lines[i];
    const at = base + i;

    if (line.trim() === '') { i++; continue; }

    if (line.startsWith(FENCE)) {
```

```
      const lang = line.slice(3).trim();
      const body: string[] = [];
      i++;
      while (i < lines.length && !lines[i].startsWith(FENCE)) body.push(lines[i++]);
      i++; // skip the closing fence
      out.push({ type: 'code', lang, value: body.join('\n'), line: at });
      continue;
    }

    let n = 0;
    while (line[n] === '#') n++;
    if (n >= 1 && n <= 6 && line[n] === ' ') {
      out.push({
        type: 'heading',
        level: n as 1 | 2 | 3 | 4 | 5 | 6,
        children: parseInline(line.slice(n + 1).trim()),
        line: at,
      });
      i++;
      continue;
    }

    if (line === '---' || line === '***' || line === '___') {
      out.push({ type: 'hr', line: at });
      i++;
      continue;
    }

    if (line.startsWith('>')) {
      const inner: string[] = [];
      while (i < lines.length && lines[i].startsWith('>')) {
        inner.push(lines[i].slice(1).trimStart());
        i++;
      }
      out.push({ type: 'quote', children: parseBlocks(inner.join('\n'), at), line: at });
      continue;
    }

    // lists and paragraphs follow the same pattern: collect lines, then build a node
    const para: string[] = [];
    while (i < lines.length && lines[i].trim() !== '' && !startsBlock(lines[i])) {
      para.push(lines[i++].trim());
    }
    out.push({ type: 'paragraph', children: parseInline(para.join(' ')), line: at });
  }
  return out;
}
```

`startsBlock(line)` is a small helper that returns true for a fence, heading, quote, rule or list marker, so a paragraph stops where another block begins. Lists are left as the exercise: detect `-` or `1.`, collect indented continuation lines per item, and call `parseBlocks` on each item.

Done when: the fixtures from Stage 0 produce sensible trees and nothing throws.

5. Stage 2: the inline parser

Goal: parse the text inside a paragraph or heading.

Steps

1. Scan the string one character at a time with a text buffer.
2. On a special character (`` ` ``, `*`, `_`, `[`), look for its closing partner.
3. If a partner is found, flush the buffer, build a node, and recurse into the inside for nested styles.
4. If not, treat the character as plain text.
5. Handle code spans first, because nothing inside them should be parsed.

```
function parseInline(s: string): Inline[] {
  const out: Inline[] = [];
  let buf = '';
  const flush = () => {
    if (buf) { out.push({ type: 'text', value: buf }); buf = ''; }
  };
  let i = 0;

  while (i < s.length) {
    if (s[i] === '`') {
      const end = s.indexOf('`', i + 1);
      if (end > 0) {
        flush();
        out.push({ type: 'code', value: s.slice(i + 1, end) });
        i = end + 1;
        continue;
      }
    }
    if (s.startsWith('**', i)) {
      const end = s.indexOf('**', i + 2);
      if (end > 0) {
        flush();
        out.push({ type: 'strong', children: parseInline(s.slice(i + 2, end)) });
        i = end + 2;
        continue;
      }
    }
    if (s[i] === '*' || s[i] === '_') {
      const end = s.indexOf(s[i], i + 1);
      if (end > 0) {
        flush();
        out.push({ type: 'em', children: parseInline(s.slice(i + 1, end)) });
        i = end + 1;
        continue;
      }
    }
    if (s[i] === '[') {
```

```
      const mid = s.indexOf('](', i);
      const close = s.indexOf(')', mid);
      if (mid > 0 && close > 0) {
        flush();
        out.push({
          type: 'link',
          href: s.slice(mid + 2, close),
          children: parseInline(s.slice(i + 1, mid)),
        });
        i = close + 1;
        continue;
      }
    }
    buf += s[i++];
  }
  flush();
  return out;
}
```

This version is deliberately naive. It will mishandle `snake_case_words` (the underscore reads as emphasis) and nested brackets. Fix those after the basics work. The CommonMark spec describes a proper delimiter-stack algorithm, which is the upgrade path.

Done when: `**bold with *italic* inside**` and `` `*not italic*` `` both come out right.

6. Stage 3: the renderer

Goal: walk the tree and produce an HTML string.

Steps

1. Write `esc()` and use it on every piece of user text. This is your protection against injected HTML.
2. Write `renderInline` and `renderBlock` as one `switch` each, with no default fallthrough, so TypeScript warns you about any node type you forget.
3. Check link targets with an allow list.
4. Add `data-line` to block elements.

```
const esc = (s: string) =>
  s.replace(/&/g, '&amp;')
   .replace(/</g, '&lt;')
   .replace(/>/g, '&gt;')
   .replace(/'/g, '&#39;');

const safeUrl = (u: string) =>
  ['http://', 'https://', 'mailto:', '#', '/'].some(p => u.toLowerCase().startsWith(p)) ? u :
'#';

function renderInline(nodes: Inline[]): string {
  return nodes.map(n => {
    switch (n.type) {
```

```
      case 'text':   return esc(n.value);
      case 'em':     return `<em>${renderInline(n.children)}</em>`;
      case 'strong': return `<strong>${renderInline(n.children)}</strong>`;
      case 'code':   return `<code>${esc(n.value)}</code>`;
      case 'link':   return `<a href='${esc(safeUrl(n.href))}'>${renderInline(n.children)}</a>`;
    }
  }).join('');
}

function renderBlock(b: Block): string {
  const at = `data-line='${b.line}'`;
  switch (b.type) {
    case 'heading':   return `<h${b.level} ${at}>${renderInline(b.children)}</h${b.level}>`;
    case 'paragraph': return `<p ${at}>${renderInline(b.children)}</p>`;
    case 'hr':        return `<hr ${at}>`;
    case 'quote':     return `<blockquote ${at}>${renderBlocks(b.children)}</blockquote>`;
    case 'code': {
      const cls = b.lang ? ` class='language-${esc(b.lang)}'` : '';
      return `<pre ${at}><code${cls}>${esc(b.value)}</code></pre>`;
    }
    case 'list': {
      const tag = b.ordered ? 'ol' : 'ul';
      const items = b.items.map(item => {
        const tight = item.length === 1 && item[0].type === 'paragraph';
        const inner = tight ? renderInline((item[0] as any).children) : renderBlocks(item);
        return `<li>${inner}</li>`;
      }).join('');
      return `<${tag} ${at}>${items}</${tag}>`;
    }
  }
}

const renderBlocks = (bs: Block[]) => bs.map(renderBlock).join('\n');
const render = (src: string) => renderBlocks(parseBlocks(src));
```

Try it: `render('# Hi\n\nSome **bold** text')` returns an `h1` followed by a `p` containing a `strong`.

Done when: the output for every fixture matches what a reference renderer (such as the CommonMark reference implementation) produces, ignoring whitespace.

7. Markdown rules and their HTML mapping

Use this chapter as the checklist for what the engine must support, and as the source for your unit tests: one test per row.

Block elements

Markdown	HTML	Notes
`# Title` up to `###### Title`	`h1` to `h6`	A space after the hashes is required

Markdown	HTML	Notes
Text, then `===` or `---` underneath	`h1` or `h2`	Setext headings, add in v2
Lines of text separated by a blank line	`p`	Consecutive lines join into one paragraph
Two trailing spaces, or a backslash, at line end	`br`	Hard line break
`> quote`	`blockquote`	Can contain any other block
`- item`, `* item` or `+ item`	`ul` with `li`	Marker choice does not matter
`1. item`	`ol` with `li`	Add a `start` attribute if the first number is not 1
`- [ ] task` or `- [x] task`	`li` with a disabled checkbox `input`	GFM extension
Fenced code with a language tag	`pre` with `code class='language-x'`	Contents are escaped, never parsed
Four-space indented text	`pre` with `code`	Indented code block
`---`, `***` or `___` on its own line	`hr`	Three or more characters
Pipe-separated rows with a dashes row under the header	`table`, `thead`, `tbody`, `tr`, `th`, `td`	GFM extension, colons in the dashes row set alignment

A table looks like this in source:

```
| Name | Role |
| ---- | ---- |
| Hashi | Mascot |
```

Inline elements

Markdown	HTML	Notes
`*text*` or `_text_`	`em`	
`**text**` or `__text__`	`strong`	
`~~text~~`	`del`	GFM extension
`` `code` ``	`code`	Contents escaped, not parsed
`[label](https://x.dev)`	`a href`	Optional title in quotes after the URL
`![alt](pic.png)`	`img src alt`	Same syntax as a link, with a leading `!`
`<https://x.dev>`	`a href`	Autolink

Markdown	HTML	Notes
Backslash before punctuation, such as `\*`	The literal character	Escapes the special meaning
`&` and other entities	The character	Decode or pass through

Characters that must be escaped in output

Character	Output as
`&`	`&`
`<`	`<`
`>`	`>`
`'` inside single-quoted attributes	`'`

8. Stage 4: the split-view shell

Goal: two panes, live preview, shared scroll.

Steps

1. Lay out two equal panes and a status bar.
2. On every edit, re-render after a short debounce so fast typing does not queue work.
3. Replace the preview contents in one assignment.
4. Sync scroll, first by percentage, then by `data-line`.
5. Add the keybindings from your design: `^S` save, `^P` preview, `^\` swap panes, `:q` quit.

```
let timer: number | undefined;

editor.addEventListener('input', () => {
  clearTimeout(timer);
  timer = window.setTimeout(() => {
    preview.innerHTML = render(editor.value);
  }, 30);
});

editor.addEventListener('scroll', () => {
  const range = editor.scrollHeight - editor.clientHeight;
  const ratio = range > 0 ? editor.scrollTop / range : 0;
  preview.scrollTop = ratio * (preview.scrollHeight - preview.clientHeight);
});
```

Percentage sync drifts when the preview has tall code blocks or images. The better version finds the first block whose `data-line` is at or above the editor's top visible line and scrolls the preview to that element.

If your editor is modal like the design (INSERT in the status bar), keep a small state machine: a `mode` variable, and a key handler that checks `mode` before deciding whether a key is text or a command.

Done when: typing updates the preview within one frame at normal speed, and scrolling either pane keeps the other roughly aligned.

9. Stage 5: files and command line

Goal: `mdv notes.md` opens a file, and saving never loses data.

Steps

1. Read `process.argv[2]` as the path. Missing file means an empty buffer that will be created on first save.
2. Save atomically: write to a temp file in the same folder, then rename over the original.
3. Track a `dirty` flag and show it in the title and status bar.
4. On quit with unsaved changes, ask first.
5. Watch the file, and if it changes on disk while you have edits, warn instead of silently overwriting.

```
import { existsSync, readFileSync, writeFileSync, renameSync } from 'node:fs';

const path = process.argv[2] ?? 'untitled.md';
const text = existsSync(path) ? readFileSync(path, 'utf8') : '';

function save(path: string, content: string) {
  const tmp = `${path}.tmp`;
  writeFileSync(tmp, content, 'utf8');
  renameSync(tmp, path); // atomic on the same filesystem
}
```

Normalize CRLF to LF when loading, and write back the style the file originally used so diffs stay clean.

Done when: you can open, edit, save and reopen a file with no changes to untouched lines.

10. Stage 6: PDF export

Goal: a clean PDF of the preview from one keystroke.

Steps

1. Add a print stylesheet: white background, dark text, no panes or status bar.
2. Use your shell's print-to-PDF: `webContents.printToPDF` in Electron, or `page.pdf` in headless Chromium.
3. Control page breaks so headings are not stranded at the bottom of a page.
4. Wrap long code lines instead of letting them run off the page.

```
@media print {
  .editor, .status-bar { display: none; }
  body { background: white; color: #111; }
  h1, h2, h3 { break-after: avoid; }
```

```
  pre, blockquote { break-inside: avoid; white-space: pre-wrap; }
}
```

Done when: a long fixture exports to a PDF with no clipped code and no orphaned headings.

11. Stage 7: testing and hardening

Goal: trust the engine.

Steps

1. One unit test per row in chapter 7.
2. Import the CommonMark spec examples (they are published as JSON with markdown input and expected HTML) and track how many pass. Do not aim for 100 percent on day one.
3. Fuzz the parser with random strings and check it never throws or loops forever.
4. Time a 1 MB document and keep re-render fast enough for typing.

```
test('heading', () => {
  expect(render('# Hi')).toBe("<h1 data-line='0'>Hi</h1>");
});

test('code span is not parsed', () => {
  expect(render('`*x*`')).toContain('<code>*x*</code>');
});
```

If typing slows down on large files, the fix is incremental parsing: re-parse only the blocks near the edit and reuse the rest.

12. Stage 8: polish and release

Steps

1. Export the Hashi icon in every required size and package it as `.icns` for macOS and `.ico` for Windows.
2. Finish the dark theme, then derive a light one from the same variables.
3. Write the README around the keystroke cheat sheet, since that is your product.
4. Package for each platform and test a clean install on a machine that has never seen the project.

13. Common pitfalls

- **Parsing inside code.** Always handle code spans and fences before anything else.
- **Skipping escaping.** Unescaped text in the preview is an injection hole. Escape everything, allow-list URLs.
- **Re-rendering the whole preview on every key.** Debounce, and later render only what changed.
- **Losing the caret on re-render.** The editor pane must never be re-created, only the preview.
- **Coupling the engine to the window.** If `parse` imports anything from the shell, move it out.

- **Chasing full CommonMark too early.** Get the common 90 percent working end to end first, then close gaps using the spec examples.

14. Suggested next steps

1. Finish Stage 0 and write down the stack.
2. Implement the block parser for headings and paragraphs only, and render them.
3. Wire that into the two panes, so you have a working editor within the first days.
4. Grow the engine one rule at a time, in the order of chapter 7.